InsightSerenity API Gateway

Enterprise-grade API Gateway for the InsightSerenity Platform, providing centralized traffic management, security, monitoring, and distributed tracing capabilities.

Features

Core Capabilities

- **Centralized Traffic Management**: All API requests flow through the gateway before reaching backend services
- **Service Discovery**: Support for static, Consul, and etcd-based service discovery
- **Load Balancing**: Multiple strategies including round-robin, least-connections, random, and weighted
- **Circuit Breaker Pattern**: Automatic failure detection and recovery with configurable thresholds
- **Rate Limiting**: Sophisticated rate limiting at global, endpoint, user, and tenant levels
- **Authentication & Authorization**: JWT-based authentication with role and permission validation
- **Multi-Tenancy**: Support for subdomain, header, and path-based tenant isolation
- **Distributed Tracing**: OpenTelemetry integration with automatic trace propagation
- **Metrics & Monitoring**: Prometheus metrics with Grafana dashboards
- **Request/Response Transformation**: Modify headers and payloads in flight
- **Caching**: Redis-based caching with configurable TTL per endpoint
- **WebSocket Support**: Proxy WebSocket connections with authentication
- **API Documentation**: Auto-generated OpenAPI/Swagger documentation

Security Features

- JWT token validation and refresh
- IP whitelisting
- CORS configuration
- Security headers (Helmet.js)
- Request sanitization
- SQL injection protection
- XSS protection
- CSRF protection

Monitoring & Observability

- Health check endpoints (liveness, readiness, startup)

- Prometheus metrics endpoint
- Distributed tracing with OpenTelemetry
- Structured logging with Winston
- Service health monitoring
- Circuit breaker metrics
- Performance metrics

Architecture

```
┌─────────────────────┐
│   Client Apps   │
└─────────────────────┘
          │
      ┌───▼───┐
      │ Gateway │ ◄──── Rate Limiting, Auth, Tracing
      └───────┘
          │
      ┌───▼─────────────┐
      │ Service Discovery │ ◄──── Consul/etcd
      └─────────────────┘
          │
      ┌───▼───┬───────┬─────────┐
      │       │       │
  ┌───▼───┐ ┌───▼───┐ ┌───▼───┐
  │Admin│ │Customer│ │Other │
  │Server│ │Services│ │Service│
  └───────┘ └───────┘ └───────┘
```

Quick Start

Prerequisites

- Node.js 18+
- Redis (for caching and rate limiting)
- Docker (optional)
- Kubernetes (optional)

Installation

```bash
```

```bash
# Navigate to gateway directory
cd servers/gateway

# Install dependencies
npm install

# Copy environment configuration
cp .env.example .env.development

# Configure your environment variables
nano .env.development
```

Running Locally

```bash
# Development mode
npm run start:dev

# Production mode
NODE_ENV=production npm start

# With specific configuration
NODE_ENV=staging GATEWAY_PORT=3000 npm start
```

Docker Deployment

```bash
# Build Docker image
docker build -t insightserenity/api-gateway .

# Run container
docker run -d \
  --name api-gateway \
  -p 3000:3000 \
  -p 9090:9090 \
  --env-file .env.production \
  insightserenity/api-gateway
```

Kubernetes Deployment

```bash
```

```
# Deploy to Kubernetes
kubectl apply -f ../../kubernetes/gateway/

# Check deployment status
kubectl rollout status deployment/api-gateway -n insightserenity

# View logs
kubectl logs -f deployment/api-gateway -n insightserenity
```

Configuration

Environment Variables

Key configuration options (see `.env.example` for complete list):

```bash
# Server Configuration
GATEWAY_PORT=3000
GATEWAY_HOST=0.0.0.0
GATEWAY_WORKERS=0  # 0 = number of CPU cores

# Service URLs
ADMIN_SERVER_URL=http://admin-server:4001
CUSTOMER_SERVICES_URL=http://customer-services:4002

# Authentication
JWT_SECRET=your-secret-key
JWT_EXPIRES_IN=1h

# Redis Cache
REDIS_HOST=localhost
REDIS_PORT=6379

# Rate Limiting
RATE_LIMIT_ENABLED=true
RATE_LIMIT_MAX_REQUESTS=100

# Tracing
TRACING_ENABLED=true
TRACING_ENDPOINT=http://jaeger:4318/v1/traces
```

Routing Rules

Configure routing in `config/environments/base.config.js`:

```javascript
routing: {
  rules: [
    {
      name: 'admin-routes',
      path: '/api/admin',
      target: 'admin-server',
      methods: ['*'],
      stripPath: false,
      loadBalancing: 'round-robin'
    },
    {
      name: 'customer-routes',
      path: '/api',
      target: 'customer-services',
      methods: ['*'],
      stripPath: false,
      loadBalancing: 'least-connections'
    }
  ]
}
```

API Endpoints

Health Checks

- `GET /health` - Overall health status
- `GET /health/live` - Liveness probe
- `GET /health/ready` - Readiness probe
- `GET /health/detailed` - Detailed health information
- `GET /health/services` - Backend services health

Metrics

- `GET /metrics` - Prometheus metrics

Documentation

- `GET /docs` - Swagger UI documentation

Admin

- `GET /admin` - Admin interface (requires authentication)

Monitoring

Prometheus Metrics

The gateway exposes metrics at `http://localhost:9090/metrics`:

- `gateway_http_request_duration_seconds` - Request duration histogram
- `gateway_http_requests_total` - Total request counter
- `gateway_circuit_breaker_state` - Circuit breaker state
- `gateway_rate_limit_hits_total` - Rate limit hits
- `gateway_cache_hits_total` - Cache hit rate
- `gateway_service_health` - Backend service health

Distributed Tracing

Traces are sent to configured OpenTelemetry collector. View traces in Jaeger UI:

```
http://localhost:16686
```

Logging

Structured logs are written to:

- Console (development)
- Files (production): `/var/log/gateway/`
- External logging service (if configured)

Development

Running Tests

```bash
# Run all tests
npm test

# Run with coverage
npm run test:coverage

# Run integration tests
npm run test:integration

# Run load tests
npm run test:load
```

Code Quality

```bash
# Lint code
npm run lint

# Fix linting issues
npm run lint:fix

# Type checking
npm run type-check
```

Debugging

```bash
# Run with debug logs
DEBUG=* npm run start:dev

# Run with Node inspector
node --inspect server.js
```

Production Deployment

Performance Tuning

1. **Clustering**: Set `GATEWAY_WORKERS` to number of CPU cores
2. **Connection Pooling**: Configure `KEEP_ALIVE_TIMEOUT` and `HEADERS_TIMEOUT`
3. **Caching**: Enable Redis caching for frequently accessed endpoints
4. **Compression**: Enable gzip compression for responses
5. **Rate Limiting**: Configure appropriate limits per endpoint

Security Hardening

1. **JWT Keys**: Use RS256 algorithm with public/private keys
2. **IP Whitelisting**: Enable for admin endpoints
3. **HTTPS**: Terminate SSL at gateway or load balancer
4. **Security Headers**: Configure Helmet.js options
5. **Input Validation**: Enable request validation schemas

High Availability

1. **Multiple Instances**: Deploy multiple gateway instances
2. **Health Checks**: Configure proper liveness and readiness probes
3. **Circuit Breakers**: Tune thresholds for your services
4. **Failover**: Configure backup service URLs
5. **Monitoring**: Set up alerts for critical metrics

Troubleshooting

Common Issues

1. **Connection Refused**
   - Check if backend services are running
   - Verify service URLs in configuration
   - Check network connectivity
2. **Authentication Failures**
   - Verify JWT secret matches across services
   - Check token expiration
   - Validate user permissions
3. **Rate Limit Exceeded**
   - Review rate limit configuration
   - Check for legitimate traffic spikes
   - Consider increasing limits
4. **High Latency**
   - Check circuit breaker status
   - Review service health
   - Analyze trace data

Debug Commands

```bash
```

```
# Check gateway health
curl http://localhost:3000/health

# View circuit breaker status
curl http://localhost:3000/admin/circuit-breakers

# Check service discovery
curl http://localhost:3000/admin/services

# View current configuration
curl http://localhost:3000/admin/config
```

Contributing

1. Fork the repository
2. Create a feature branch
3. Commit your changes
4. Push to the branch
5. Create a Pull Request

License

MIT License - see LICENSE file for details

Support

For issues and questions:

- GitHub Issues: InsightSerenity Issues
- Documentation: InsightSerenity Docs
- Email: support@insightserenity.com